

Get the GIST: An Interactive Toolkit to Support Generative Design through Metaheuristic Optimization

Megan Hofmann

Human Computer Interaction Institute, Carnegie Mellon University
Pittsburgh, PA, USA

ABSTRACT

Generative design tools afford designers new ways of creating complex objects from 3D models to machine knittable textures. However, implementing the optimization algorithms that make generative design possible requires the rare combination of programming and domain expertise. I present the Generative Interactive Synthesis Toolkit (GIST) to simplify the implementation of generative design tools in novel domains. GIST factors common optimization algorithms into elements of an extensible library and structures optimization tasks around objectives and tactics specified by domain experts in a simple GUI. This moves the burden of domain expertise from programmers to domain experts. I demonstrate GIST in three unique domains: machine knitting, cookie recipes, and tactile maps for blind users. These show the versatility of GIST's structure.

CCS CONCEPTS

• Human-centered computing → Interactive systems and tools.

KEYWORDS

fabrication; optimization; metaheuristics; generative design

ACM Reference Format:

Megan Hofmann. 2021. Get the GIST: An Interactive Toolkit to Support Generative Design through Metaheuristic Optimization. In *The Adjunct Publication of the 34th Annual ACM Symposium on User Interface Software and Technology (UIST '21 Adjunct)*, October 10–14, 2021, Virtual Event, USA. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3474349.3477584>

1 INTRODUCTION

Generative design enables us to build new and better designs through optimization. However, generative design tools are often necessarily domain specific which makes them difficult to implement because they require collaboration between programmers and domain experts. My thesis details the implementation and application of the Generative Interactive Synthesis Toolkit. GIST enables domain experts and programmers to collaboratively build domain specific tools, and unlock new domains for digital fabrication.

GIST enables programmers to implement generative design tools and enables domain experts to embed their expertise into the optimization process. There are numerous optimization toolkits for

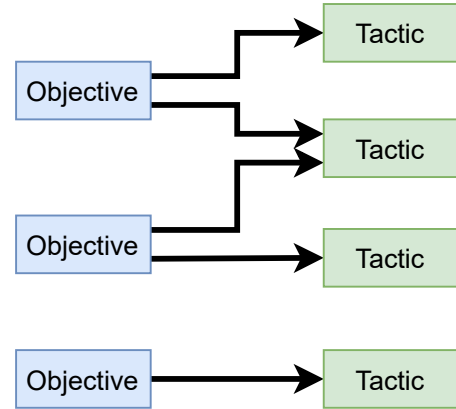


Figure 1: Mapping objectives to tactics expresses which tactics are most likely to improve which objectives.

various domains (e.g., user interface layouts [5, 6], 3D modeling [3, 13], image editing [15], and machine knitting [9]). While each of these systems enable programming generative design systems, they do so in limited domains or by using only one of many possible optimization algorithms (e.g., topology optimization [3, 13]). GIST builds abstractions and concepts from these prior systems but uniquely enables programmers and designers to collaboratively build generative design tools in the domains of their choosing.

2 HEURISTIC MAPS

$$\alpha_{o \rightarrow t}(S_t) = \frac{\sum_{s, s' \in S_t} o(s') - o(s)}{|S_t|} \quad (1)$$

GIST's The search process is guided by the relationship between designer-ranked domain-specific *objectives* and *tactics*, small modifications to a design that may help improve objective scores. Pairing objectives and tactics specifies *heuristics*; general rules for how to improve a particular objective. I call the mapping of weighted objectives to tactics a *heuristic map*.

Heuristic Maps are the core of GIST that enable designers to specify different goals and the means of meeting them. Internally, heuristic maps are a weighted set of objectives and a weighted mapping of objectives to tactics (Figure 1). Each objective, o , in the objective set, O , has a weight, β_o , that denotes its value. Each objective given a design, d , will return a value between 0 (performs poorly) and 1 (performs perfectly). A tactic, t , in the set of tactics, T , modifies a design to produce a new design. Each tactic can be paired with multiple objectives and each objective-tactic pair has a *heuristic weight*, $\alpha_{o \rightarrow t}$. Heuristic weights measure how well a tactic

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST '21 Adjunct, October 10–14, 2021, Virtual Event, USA

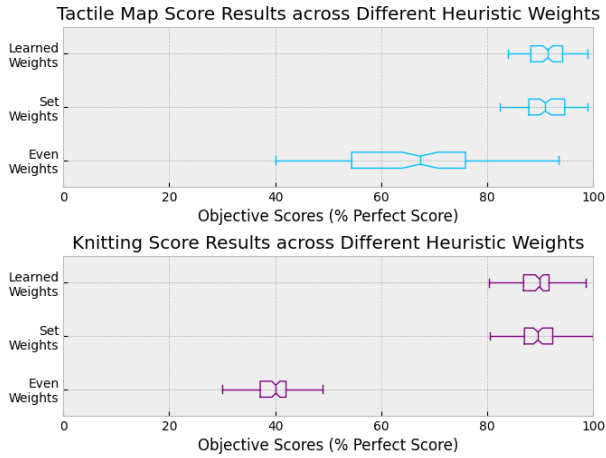
© 2021 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-8655-5/21/10.

<https://doi.org/10.1145/3474349.3477584>

Table 1: Summary of Dependent T tests comparing the effect of learned heuristic weights, domain-expert set weights, and even weights across all tactic objective pairings.

Domain	Conditions Compared	Scores (% Perfect Score)		Time to Convergence (Seconds)	
		T-Value	p-Value	T-Value	p-Value
Tactile Maps	Learned vs Domain-Expert's Weights	-1.73	>0.1	1.22	>0.1
	Learned vs Even Weights	2.69	<0.1	1.28	>0.1
	Domain-Expert's vs Even Weights	2.60	<0.1	106.90	<.0001
	Learned vs Domain-Expert's Weights	0.57	>0.1	0.79	>0.1
Knitted Textures	Learned vs Domain-Expert's Weights	13.31	<0.001	13.36	<0.001
	Learned vs Even Weights	25.00	<0.0001	34.55	<0.0001
	Domain-Expert's vs Even Weights				
	Learned vs Domain-Expert's Weights				

**Figure 2: Learned heuristic weights outperform even weights and are comparable to expert weights.**

is expected to improve a particular objective score. Each GIST's objective function is the the weighted sum of the objective-scores.

In many domains the relationship between tactics and objectives may be unclear. GIST can learn these weights from a small sample of designs. GIST estimates heuristic weights by calculating the expected value of objective-score improvements given a large sample of design pairs that differ only by the application of a tactic. GIST generates these pairs starting with a set of designs provided by a designer which are scored across all objectives. Then tactics are applied to each to generate new designs that differ by a tactic. The difference between objective scores is a measure of a tactics' efficacy for each objective (i.e., the heuristic weight). The measure improves by repeating the process, applying tactics to the generated samples. That is, the heuristic weight, $\alpha_{o \rightarrow t}$, between a tactic, t , and an objective, o , is the average difference in scores between paired samples s and s' , where s' is the result of applying t to s , in a large set of sample pairs S (Equation 1).

The quality of heuristic weights should have significant effects on GIST's performance. To evaluate the differences between the heuristics learned using this algorithm and those created by a designer I generated three sets of heuristic weights: the *learned weights*; *domain-expert's weights*; and *even weights* where each tactic is paired to each objective with a heuristic weight of 1. In each domain, I generated 100 sets of objectives with random weights between 0 and 10. I built heuristic maps that combined these objectives with heuristic weights across each experimental condition.

Across both test domains that, the learned weights consistently converged on higher scores than even weights and were comparable to scores produced by domain expert's weights. The domain-expert weights also outperformed even weights, demonstrating that the addition of domain expertise is what distinguishes optimizers that converge on lower local maxima or higher local maxima. I summarize these results in Table 1 and Figure 2.

3 GIST OPTIMIZERS

Optimization starts with a set of designs provided by the designer. Next, this design is scored based on the objectives in the designer's heuristic map. GIST's objective function is the weighted sum of these objectives scores given a particular design. Given this score, a *design selector* from the GIST library chooses which design to modify in the next iteration. This design is compared against some pluggable *stopping criteria* (e.g., meets threshold score, score converges) which determines if the result should be returned. If the search continues, a pluggable *tactic selector* uses information about the objectives' performance and the heuristic weights in the heuristic map to choose how to modify the design. The selected tactic is applied and the new design is added to a pool of designs to consider in the next iteration. This process will stochastically search for a design that best meets the designers specified goals.

The pluggable elements of the GIST library can be combined into a variety of metaheuristic search algorithms. Metaheuristics determine how to apply domain specific heuristics. GIST's library of selectors use information from the population of design iterations and the designer's heuristic map to guide the search process. The

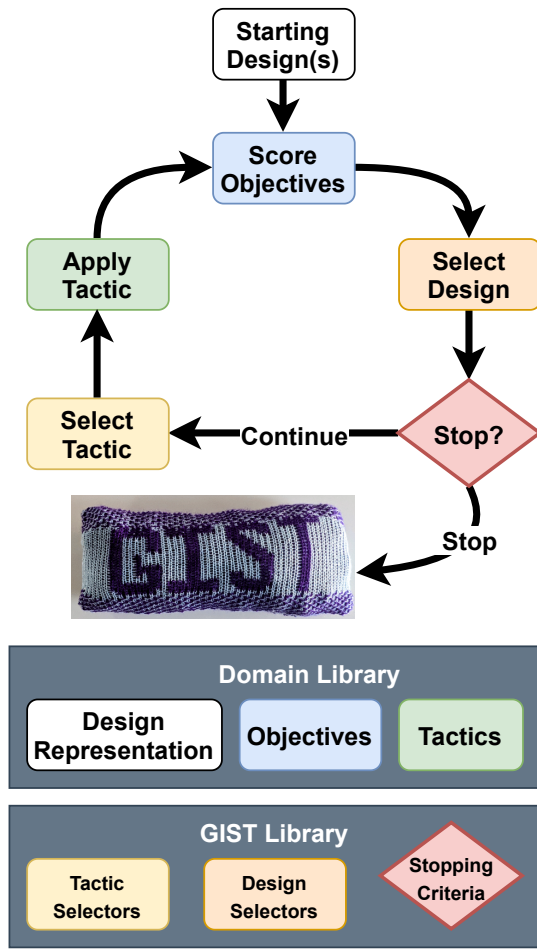


Figure 3: The flow of a GIST optimizer

existing library provides design and tactic selectors derived from common algorithms: Ant Colony Optimization [4], Guided Local Search [17], Iterated Local Search [11], simulated annealing [16], Tabu Search [7], Variable Neighborhood Search [14].

For example, combining a design selector that selects high performing designs and a tactic selector that chooses the tactic that is most highly weighted on low performing objectives performs simple hill-climbing. Switching out the tactic selector to instead choose tactics that in previous iterations improved the objective score will perform Ant Colony Optimization [4]. Alternatively, selecting designs randomly with an increasing bias towards high performing designs after each iteration will conduct simulated annealing. Because swapping out different components is easy for programmers, experimenting with different metaheuristics is trivial in GIST.

4 PROGRAMMING FOR GIST

To build a optimizer-interface for designers, the programmer must implement three components: (1) a data structure that represents designs in the domain (i.e., *design representations*), (2) *objectives* that evaluate properties of a design representation and return a value between 0 (i.e., poor performance) and 1 (i.e., good performance),

Cookie GIST

Configuration

Number of Cookies 24

Objectives

Chewy Add Objective

Importance	Objective	Objective Parameters	Remove Objective
Weight: 1.0	Crisp_0		Remove
Weight: 2	Chewy_0		Remove

Tactics

Learn Tactic Weights

Submit

(a) Designers apply weights to objectives from a drop down list.

Tactics

Set Custom Tactic Weights

IncreaseWhiteSugar Add Tactic

IncreaseBrownSugar 1

Tactic's Importance to Objective	Objective
1.0	Chewy_0

IncreaseWhiteSugar 2

Submit

(b) Designers add tactics and apply weights to relevant objectives.

Cookie GIST Results

Update Results

Result 1 (Total Score: 1.9375):

Importance	Objective	0 ≤ Score ≤ 1	Weighted Score (I*S)
1	Crisp_0	0.4375	0.4375
2	Chewy_0	0.75	1.5

Accept Result 1

Result 2 (Total Score: 1.60416)

Result 3 (Total Score: 1.54166)

(c) GIST shows a list of high scoring results for review.

Figure 4: GIST has a three part interface. First, they set objective weights (a) then optionally define heuristic weights (b). Next, they browse their optimized results(c).

and (3) *tactics* that modify design representations. Given these elements, a designer can mix and match objectives and tactics in a heuristic map to guide the GIST optimizer.

5 DESIGNING WITH GIST

Designers construct heuristic maps and view the optimization results through a graphical user interface generated by GIST (Figure 4). While this interface is very simple, it demonstrates how this toolkit effectively breaks up optimization into structures that can be easily manipulated by designers. Giving the designers the ability to not only define the objectives for a specific search but to related objectives and tactics to create heuristics gives them substantial influence over the tool without programming.

6 DEMONSTRATIONS

GIST gives designers more control over generative design systems. There are numerous domains where domain expertise is critical to guiding an effective optimization, but few domain experts will have the requisite programming skills. Further, GIST is designed to be applicable to a wide variety of domains. As such, I selected three



Figure 5: Three applications of GIST: chocolate chip cookie recipe optimization, generation of customized tactile maps for blind users, and generation of colorful machine knittable objects.

demonstrative domains that use different types of design representations and position different types of users as expert designers.

An early iteration of GIST was used to build *KnitGIST* [9] and it uses hand knitters' domain expertise to generate functional machine knitting patterns. In *KnitGIST*, estimations of key physical properties (e.g., curl, elasticity, shrinkage) are set as objectives and modifications of different stitches are used as tactics. As GIST evolved, I included a simple demonstrative domain of chocolate chip cookies which use rules of thumb from a cooking show [1] to optimize recipes by adjusting ingredient amounts. The show explains how ingredients effect different types cookie textures (e.g., chewy, crispy, cakey). The cookie objectives estimate these textures and tactics modify the key ingredient amounts. Finally, related to my other lines of research on fabrication and accessibility [2, 8, 10, 12], I have built an optimizer for 3D printable tactile maps that can help blind users navigate outdoor spaces. The tactile map objectives represent types of locations (e.g., specific roads and buildings, amenities). The tactics represent ways of representing this information (e.g., embossed path ways, icons). The optimizer uses a blind user's ranking of different types of information to adjust the map to include important information in the most readable way.

7 LIMITATIONS

GIST has a few key limitations that open promising opportunities for future work. First, while I have used GIST in three domains an extensive study of development with GIST by real teams of programmers and designers remains an area of future work. Second, while I have greatly reduced requirements that programmers have domain expertise, programmers still must build domain specific design representations, objectives, and tactics by collaborating with domain experts. There is an opportunity to build graphic interfaces that help designers specify these structures without programming.

8 CONCLUSION

My thesis contributes the Generative Interactive Synthesis Toolkit (GIST) for building generative design tools in unique domains with metaheuristic optimization methods. Unlike other toolkits, GIST separates the roles of programmers (people who implement optimization algorithms) and designers (people with critical domain expertise). Designers build heuristic maps that map their domain specific objectives to tactics for modifying designs to meet those goals. This is done in a simple user interface that GIST generate. To build an optimizer, programmers create a representation of designs in the domain, a set of objectives that measure design quality, and set of tactics that modify and improve designs. Designers and programmers can mix and match library element to implement a variety of metaheuristics.

ACKNOWLEDGMENTS

This work was funded by National Science Foundation Grants: IIS-1718651, IIS-1907337, and 2031801. This work has been supported by dedicated teams of undergraduate and PhD student collaborators at Carnegie Mellon University and the University of Washington. My thesis has been guided by the careful hands of my advisors, Jennifer Mankoff and Scott E. Hudson.

REFERENCES

- [1] Alton Brown. 2020. The Chewy Chocolate Chip Cookie Recipe. <https://altonbrown.com/recipes/the-chewy-chocolate-chip-cookie> [Online; accessed 6. Apr. 2021].
- [2] Erin Buehler, Amy Hurst, and Megan Hofmann. 2014. Coming to Grips: 3D Printing for Accessibility. In *Proceedings of the 16th International ACM SIGACCESS Conference on Computers & Accessibility* (Rochester, New York, USA) (ASSETS '14). Association for Computing Machinery, New York, NY, USA, 291–292. <https://doi.org/10.1145/2661334.2661345>
- [3] Xiang 'Anthony' Chen, Ye Tao, Guanyun Wang, Runchang Kang, Tovi Grossman, Stelian Coros, and Scott E. Hudson. 2018. Forte: User-Driven Generative Design. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (CHI '18). Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3173574.3174070>
- [4] M. Dorigo, M. Birattari, and T. Stutzle. 2006. Ant colony optimization. *IEEE Computational Intelligence Magazine* 1, 4 (2006), 28–39. <https://doi.org/10.1109/MCI.2006.329691>
- [5] James Fogarty and Scott E. Hudson. 2003. GADGET: A Toolkit for Optimization-Based Approaches to Interface and Display Generation. In *Proceedings of the 16th Annual ACM Symposium on User Interface Software and Technology* (Vancouver, Canada) (UIST '03). Association for Computing Machinery, New York, NY, USA, 125–134. <https://doi.org/10.1145/964696.964710>
- [6] Krzysztof Gajos and Daniel S. Weld. 2004. SUPPLE: Automatically Generating User Interfaces. In *Proceedings of the 9th International Conference on Intelligent User Interfaces* (Funchal, Madeira, Portugal) (IUI '04). Association for Computing Machinery, New York, NY, USA, 93–100. <https://doi.org/10.1145/964442.964461>
- [7] Fred Glover and Manuel Laguna. 1998. *Tabu Search*. Springer US, Boston, MA, 2093–2229. https://doi.org/10.1007/978-1-4613-0303-9_33
- [8] Megan Hofmann, Jeffrey Harris, Scott E. Hudson, and Jennifer Mankoff. 2016. Helping Hands: Requirements for a Prototyping Methodology for Upper-Limb Prosthetics Users. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems* (San Jose, California, USA) (CHI '16). Association for Computing Machinery, New York, NY, USA, 1769–1780. <https://doi.org/10.1145/2858036.2858340>
- [9] Megan Hofmann, Jennifer Mankoff, and Scott E. Hudson. 2020. KnitGIST: A Programming Synthesis Toolkit for Generating Functional Machine-Knitting Textures. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology* (Virtual Event, USA) (UIST '20). Association for Computing Machinery, New York, NY, USA, 1234–1247. <https://doi.org/10.1145/3379337.3415590>
- [10] Megan Hofmann, Kristin Williams, Toni Kaplan, Stephanie Valencia, Gabriella Hann, Scott E. Hudson, Jennifer Mankoff, and Patrick Carrington. 2019. "Occupational Therapy is Making": Clinical Rapid Prototyping and Digital Fabrication. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland UK) (CHI '19). Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3290605.3300544>

- [11] Helena R. Lourenço, Olivier C. Martin, and Thomas Stützle. 2003. Iterated Local Search. In *Handbook of Metaheuristics*. Springer, Boston, MA, Boston, MA, USA, 320–353. https://doi.org/10.1007/0-306-48056-5_11
- [12] Jennifer Mankoff, Megan Hofmann, Xiang 'Anthony' Chen, Scott E. Hudson, Amy Hurst, and Jeeun Kim. 2019. Consumer-Grade Fabrication and Its Potential to Revolutionize Accessibility. *Commun. ACM* 62, 10 (Sept. 2019), 64–75. <https://doi.org/10.1145/3339824>
- [13] Justin Matejka, Michael Glueck, Erin Bradner, Ali Hashemi, Tovi Grossman, and George Fitzmaurice. 2018. Dream Lens: Exploration and Visualization of Large-Scale Generative Design Datasets. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (CHI '18). Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3173574.3173943>
- [14] N. Mladenović and P. Hansen. 1997. Variable neighborhood search. *Computers & Operations Research* 24, 11 (Nov 1997), 1097–1100. [https://doi.org/10.1016/S0305-0548\(97\)00031-2](https://doi.org/10.1016/S0305-0548(97)00031-2)
- [15] Evan Shimizu, Matthew Fisher, Sylvain Paris, James McCann, and Kayvon Fatahalian. 2020. Design Adjectives: A Framework for Interactive Model-Guided Exploration of Parameterized Design Spaces. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology* (Virtual Event, USA) (UIST '20). Association for Computing Machinery, New York, NY, USA, 261–278. <https://doi.org/10.1145/3379337.3415866>
- [16] Peter J. M. van Laarhoven and Emile H. L. Aarts. 1987. Simulated annealing. In *Simulated Annealing: Theory and Applications*. Springer, Dordrecht, The Netherlands, 7–15. https://doi.org/10.1007/978-94-015-7744-1_2
- [17] Christos Voudouris and Edward P. K. Tsang. 2003. Guided Local Search. In *Handbook of Metaheuristics*. Springer, Boston, MA, Boston, MA, USA, 185–218. https://doi.org/10.1007/0-306-48056-5_7